MahaSetu AI: Seva Challenge Submission Guide

Oct 20, 2026 · @kkk

Engineering details (requirements targets, API, state machines, threat model, decisions, operations, testing) are in the tab Engineering pack.

1. Problem and solution in simple words

Maharashtra's departments run separate portals, so people repeat the same paperwork and nobody sees the whole picture. MahaSetu AI connects them without replacing them.

The problem. Each department has its own portal, data format, ID and login. As a result:

- Citizens and businesses submit the same documents again and again.
- They visit several portals or offices to track one application.
- They often do not know which approvals apply, so applications arrive incomplete and get delayed.
- Officials have no single view of applications, approvals and grievances, so bottlenecks stay hidden.
- Services are manual and reactive, with little AI to guide users or flag problems early.

Core challenge: deliver smart, citizen-friendly services across departments without replacing existing systems.

The solution in one line. An AI-enabled unified service platform, built on an interoperability layer, that lets a person apply once, track everything with one ID, and be warned about delays before they happen.

What it is not. It does not replace department systems and does not approve applications. Officers still decide.

Theme: Emerging Tech, AI-Powered Citizen Services and Smart Governance for the State.

2. Guide parameters and project status

The First-Time Problem Solver's Guide sets seven steps, each ending in a "write it down" box. Four are drafted here; three need your own evidence before submission.

Step	What the guide requires	Status for MahaSetu AI	What you must add
1. Problem	One sentence: who loses what because of what, how often. Two or three facts from affected people.	Sentence drafted (section 4). Facts not yet collected.	Interview 3 people (a citizen, an MSME owner, a clerk): how often, what it costs, what they do today.
2. Solution	One line, plus what it is made of, who uses it, what it replaces. Say what it is NOT.	Done.	Nothing.
3. Method	Observation, root cause (ask why five times), three options with two rejected, prototype, test and change. Include one failure. Name any tool used.	Options and root cause drafted. No prototype or failure yet.	Build the prototype, log one real failure and its fix, name tools such as an LLM or OCR library.
4. Impact	Baseline, result after a trial, per-person meaning, and how many others have the problem. Honest "not yet tested" beats invented numbers.	Targets only, labelled as targets.	Measure real timings in your demo: with and without the platform.
5. Government owner	The one department, the local office you can reach, the officer designation.	Department and designation drafted.	Confirm the local office and exact designation.
6. Scale	Four stages (1 site, 5 to 10, taluka or district, state), a number per stage, and who runs each site.	Drafted.	Confirm the numbers and the role at each site.

Step	What the guide requires	Status for MahaSetu AI	What you must add
7. Money	One-time cost, monthly cost, monthly benefit, who pays at each stage. Mark estimates.	Cost split drafted as estimates. Monthly benefit not worked out.	Add a monthly benefit per site against monthly cost per site.

Template rules (Seva PPT). Keep to 6 slides in the given order. Replace every grey bracketed text and delete the guidance boxes. Use simple language, facts and numbers with sources. Add photos of the prototype or field visits. Save as PDF, maximum 10 MB.

3. Slide-by-slide content (6 slides)

Slide 1. Title. MahaSetu AI (placeholder name). One line: an AI-enabled unified service platform that connects Maharashtra's existing department systems, so a citizen or business applies once, tracks everything with one ID, and gets warned before delays happen. Team and contact: to be filled.

Slide 2. The Problem.

Field	Content
What is the problem	Separate portals, formats, IDs and logins force repeat submissions and hide delays from officials.
Who is affected, where	Citizens and businesses (especially MSMEs) across Maharashtra, plus officers and senior officials.
Evidence	The official problem brief (Seva Challenge 2026). Add your 2 to 3 interview facts with sample size and date.
Current gaps	One portal per department, no single status view, no reuse of verified documents, delays found only after they happen.
Big number box	Many to 1: many portals, IDs and logins reduced to one.

Field	Content
Why it matters	Duplicate submissions, incomplete files and hidden delays cost people time and money.

Slide 3. Our Solution and Innovation.

- In brief: an AI layer over existing systems, using an API gateway, reusable connectors and a common data model. It adds one login, one tracking ID, consent-based sharing, a Marathi and English assistant, document pre-checks, parallel routing, SLA breach warnings and admin analytics.
- Input: the user logs in once, enters details once and gives consent.
- Process: the assistant lists the approvals needed, documents are pre-checked, applications route to departments in parallel, SLA timers run.
- Output: one tracking ID and status view, early delay alerts, admin analytics.
- Key USP: a consent-based AI layer that needs no system replacement and warns before delays happen. No patent filed.

	Existing approach	Our approach
Cost	Rebuild or replace systems	Plugs in through APIs
Speed	Sequential, office to office	Parallel routing and SLA alerts
Reach	A portal per department	One login, Marathi and English
Ease of use	Repeated forms	Pre-checked, entered once

Slide 4. Feasibility and Implementation.

- Stage: tick Idea now. Move to Proof of Concept when the mock-department prototype runs.
- Technology: API gateway, connectors, common data model, single sign-on with roles, consent and audit logs, rules engine and workflow with SLA timers, assistant, admin dashboard, open-source stack.
- Estimated cost (estimates, confirm with quotes), for a ₹10 lakh pilot: development and integration ₹4.5 L, cloud and AI usage ₹1.5 L, Marathi language testing ₹1 L, security review ₹1 L, field support and training ₹1 L, contingency ₹1 L.
- Timeline: 3 months, prototype with 4 mock departments. 6 months, pilot with one real department in one district. 12 months, 5 to 8 services, full analytics, handover.

- Partners: EIT&AI Department, Skills, Employment, Entrepreneurship and Innovation Department with MSInS, Industries Department, District Collector's office or municipal corporation.

Slide 5. Impact and Scalability.

- Beneficiaries, first year: target about 5,000 applicants in one pilot district (assumption).
- Saving in time: target about 30% less processing time (not yet tested; replace with your measured figure).
- Jobs or livelihoods: indirect; faster approvals help MSMEs start earlier. No direct jobs claimed.
- Social impact: apply once and in your language, guided help for rural and first-time users, fewer visits and less dependence on middlemen.
- Sustainability: runs inside existing department IT budgets by replacing repeated manual verification. Open standards avoid lock-in.
- Scale-up: 1 service cluster, then 5 to 10 services, then a district, then the state, reusing the same connectors so cost per site stays flat or falls.

Slide 6. Team and Support Required. Four members with roles: team lead for backend and integration, frontend and UX, AI assistant and validation, data and testing. Add names, qualifications, mentor and contact. Tick the support wanted: mentorship, department pilot, funding, incubation, testing and IP support. One line: Challenge support takes us from a mock-department prototype to a live pilot that gives real before-and-after data.

4. One-page project template (10 items)

This follows the workbook's one-page template. Rows marked "you add" cannot be filled honestly from here.

#	Item	Answer
01	Project name, team	MahaSetu AI (placeholder). Team: you add names.
02	Problem in one sentence	Citizens and businesses in Maharashtra lose time and repeat paperwork because each department runs a separate portal, ID and login, on every application they make.

#	Item	Answer
03	Evidence from stakeholders	You add: 2 to 3 facts from interviews with a citizen, an MSME owner and a clerk, with sample size and date. Secondary source: the official problem brief.
04	One-line solution	An AI-enabled platform over existing department systems that lets a person apply once, track with one ID and be warned before delays.
05	Methodology	Observe the repeat-submission pattern. Root cause: no shared data model, identity or tracking ID. Options: replace all systems (rejected, too costly and slow), build one new portal (rejected, adds a portal), integration layer (chosen). Then prototype with mock departments, test, and change.
06	One failure and the fix	You add: log a real failure while building, for example a connector mapping error or a retry that did not fire, and what you changed.
07	Impact, before and after	You add: measured time and number of repeat entries with and without the platform. Targets in the deck stay labelled as targets.
08	Government department and local office	EIT&AI Department or SEEID. Local contact: District Collector's office or municipal ward office. Designation to address: you confirm the exact title locally.
09	Stage-wise scale-up	1 service cluster, then 5 to 10 services, then one district, then the state. Run by a department nodal officer, then the department IT cell, then the district e-governance cell.
10	Financial details	Estimated pilot budget ₹10 lakh (see slide 4). You add: monthly benefit per site against monthly cost per site, and who pays at each stage (you or well-wishers early, department or CSR later).

5. Judges' lens and before-you-submit checks

The guide says a ₹10 lakh work order goes to the entry with four things. Two are strong here; two depend on your evidence.

Judges look for	How MahaSetu AI meets it	Strength today
A real problem with evidence	Official brief plus your interviews	Weak until you add interviews
A working demo	Mock-department prototype with the failure and retry demo	Strong once built
Fits money the government already spends	Replaces repeated manual verification and paperwork; name the department cost it reduces	Medium; name the cost
A deliverable pilot plan	Timeline, partners and cost split on slide 4	Strong

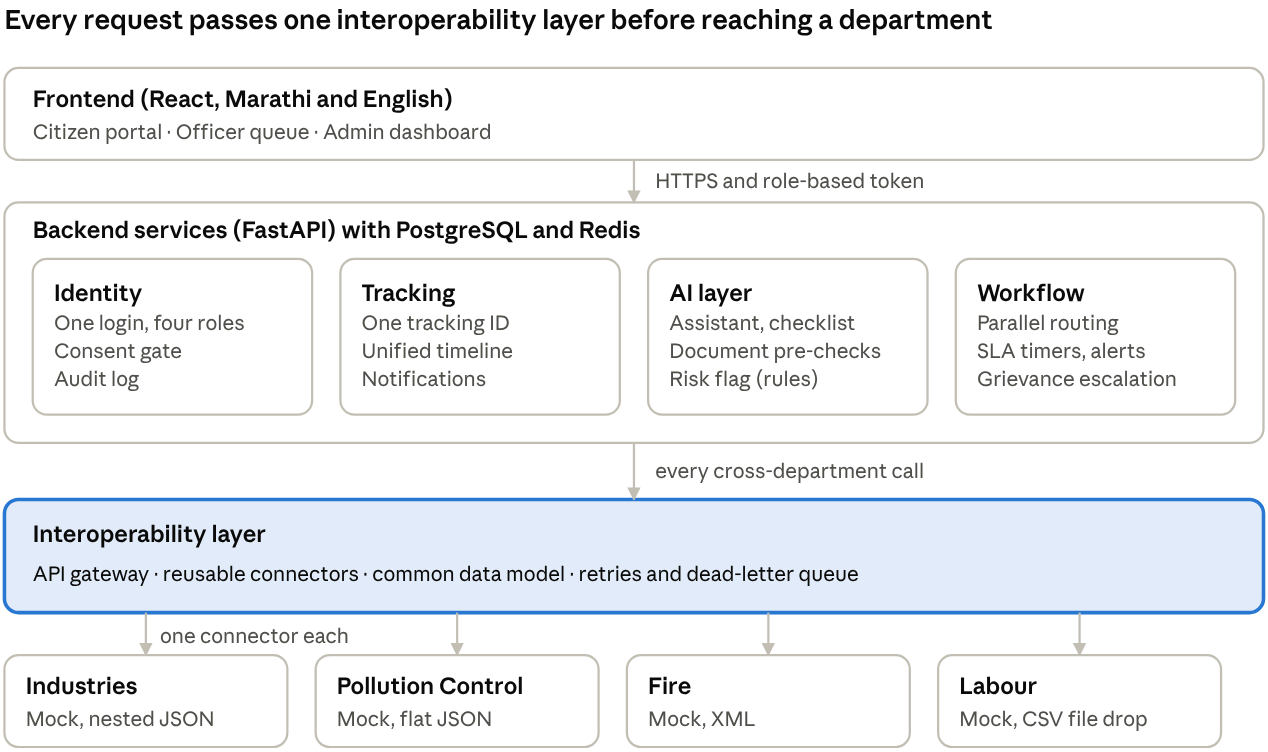
Three checks before you submit (from the guide):

- [ ] Every claim has a number or a name behind it. Remove or label any untested target.
- [ ] A stranger understands the page without you explaining.
- [ ] You can answer "what failed?" honestly.

Also check. Name every tool you used and what it did. State that departments in the demo are mocks and data is synthetic. Save the deck as a PDF of 10 MB or less, in 6 slides.

6. Implementation, architecture, user flow and demo

The platform has four layers, and every cross-department call goes through the interoperability layer, so a real department can replace a mock by changing only its connector.

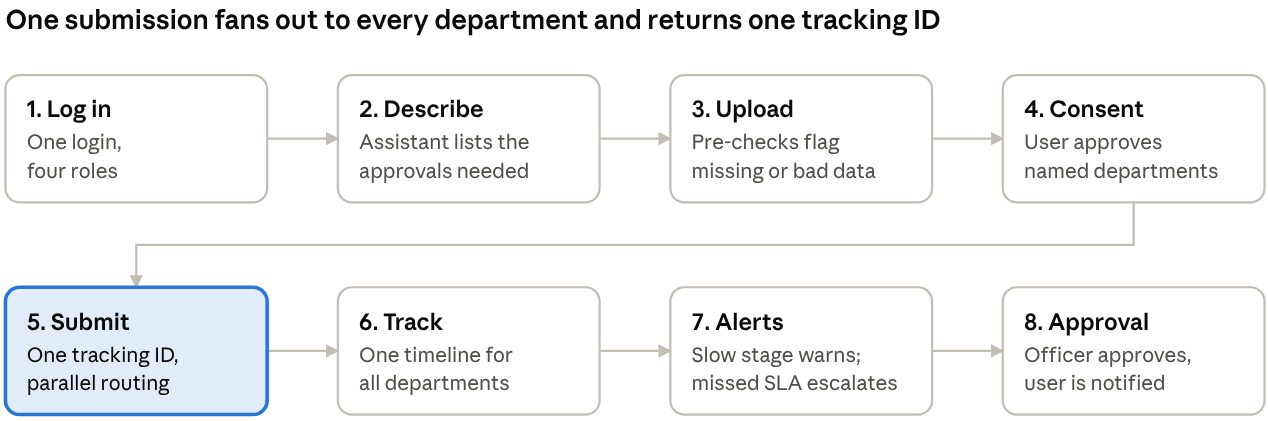


architecture · 4 layers, 4 mock departments

The frontend calls the backend over HTTPS with a role-based token; the backend reaches each department only through the gateway and its connector.

User flow.

1. Log in once (one login, four roles: citizen, business, officer, admin).
2. Describe the project; the assistant lists the approvals needed.
3. Upload documents; pre-checks flag missing or inconsistent data.
4. Give consent for the named departments.
5. Submit once; get one tracking ID.
6. Departments process in parallel; the timeline shows all of them.
7. A slow stage triggers an alert; a missed SLA raises a grievance.
8. The officer approves; the user is notified.



user flow · 8 steps

The flow reads left to right, then wraps to the second row; step 5 is the key moment, where one submission becomes one tracking ID.

Frontend and backend working flow.

Step	Frontend action	Backend call	Behind it
1	Login page	POST /auth/login	Token issued with a role
2	Project form	POST /assistant/checklist	Rules file returns required approvals
3	Upload documents	POST /validate	Missing or inconsistent fields returned
4	Consent screen	POST /consent	Consent stored and audit-logged
5	Submit	POST /applications	Tracking ID created, tasks fan out through connectors
6	Timeline page	GET /applications/{id}/timeline	Events merged from all departments
7	Officer queue	PATCH /tasks/{id}	State change, user notified
8	Admin dashboard	GET /analytics	Processing time, bottlenecks, API health from timestamps

Suggested stack. React with Marathi and English text, FastAPI, PostgreSQL, Redis for retries and the dead-letter queue, token-based roles, a state machine per department, rules first and an LLM for the assistant, Chart.js or Recharts for dashboards, Docker Compose for the demo.

Demo. The full demo story, set-up and fail-safes are in section 11.

Real portals. Live Maharashtra portals do not give students open API access; real integration needs an MoU and an authorised pilot. The prototype uses mocks modelled on real service flows, and the connector design lets a real department plug in during the pilot.

7. Gaps to fill before submitting

Seven items cannot be written from here because they need your own facts or work.

- [ ] Team names, qualifications, mentor and contact details (slides 1 and 6).
- [ ] Two or three interview facts with sample size and date (slide 2, template item 03).
- [ ] One real failure from building, and the fix (slide 3, template item 06).
- [ ] Measured before-and-after timings from your demo, replacing the labelled targets (slide 5).
- [ ] The local government office and exact officer designation (slide 4, template item 08).
- [ ] Monthly benefit against monthly cost per site, and who pays at each stage (template item 10).
- [ ] A final project name, since MahaSetu AI is a placeholder.

8. Frontend, backend and UX specification

Build three role-based apps on one backend, in Marathi and English, designed so a first-time user finishes an application without help.

UX principles

- One task per screen, with a progress bar (Describe, Upload, Consent, Submit).
- Language switch (Marathi and English) in the header on every screen; no text hard-coded.
- Plain words, large tap targets, and a layout that works on a basic phone, since many users will apply from mobile.
- Show why before asking: every document request says which department needs it.

- Never block silently: every error says what is wrong and how to fix it.
- Status in words and icons, not colour alone, for accessibility.
- The consent screen lists each department, the data shared and the purpose, with one clear Allow or Deny per department.

Screens by role

Role	Screens	Key UX detail
Citizen or business	Login, dashboard of applications, new application wizard, checklist, document upload with live checks, consent, submit confirmation, tracking timeline, grievance form, assistant chat	One tracking ID shown at the top of every application; timeline shows each department in parallel
Officer	Queue sorted by SLA due time, application detail, approve or reject with reason, request more information	SLA countdown on every row; breach risk highlighted with a label, not only colour
Admin	Dashboard (processing time, bottlenecks, SLA breaches), API health, failed-calls queue, audit log search	Retry button on failed calls; every chart has a plain-language title that states the finding

Frontend structure (React)

- Pages: Login, Dashboard, NewApplication (wizard), Tracking, OfficerQueue, AdminDashboard, ApiHealth, AuditLog.
- Shared components: LanguageSwitch, ProgressSteps, ConsentCard, DocumentUploader, StatusTimeline, SlaBadge, AssistantChat, ErrorBanner.
- State: auth token and role in memory, server data through a query library, wizard draft kept in local state so a refresh does not lose it.
- i18n: one translation file per language, keys for every label and error.
- Role routing: a route guard that sends each role to its own home screen.

Backend structure (FastAPI)

- Modules: auth, consent, applications, assistant, validation, workflow, sla, grievances, notifications, analytics, audit, connectors.

- Background worker: sends tasks to departments, retries with backoff, moves repeated failures to a dead-letter queue, and checks SLA timers every minute.
- Database tables: users (with department, so an officer sees only their own queue), applications, dept_tasks (with warned and breached flags), documents (with pre-extracted fields), consents (with expires_at), grievances, audit_log (with prev_hash and hash), api_health.
- Connector contract: each department adapter implements the same three functions, to_canonical, from_canonical and send, so a mock can be swapped for a real system without touching the rest.
- Rules files, not code: approvals checklist and SLA durations live in editable JSON so they can change without a redeploy. Delay handling is rules-based SLA breach warnings and the risk flag is a completeness rule; neither is a prediction model.
- Safety: every cross-department send checks a stored consent first and writes an audit entry; passwords are hashed; roles are enforced on the server, not only in the UI.

Error and empty states to design. Wrong login, expired session, upload too large, unsupported file type, a department API down (show "processing delayed, we are retrying"), consent denied (explain what cannot proceed), no applications yet.

9. Prompts for building

Paste these into an AI coding tool one at a time, in order. Run and test after each before sending the next. Prompt 0 goes first in every new session so the tool keeps the same context.

Prompt 0. Project context (paste first)

```
I am building MahaSetu AI, a prototype for the Seva Innovation Challenge
2026: an AI-enabled unified service platform that connects existing
Maharashtra department systems through an interoperability layer. Departments
are MOCK systems with synthetic data. Stack: React frontend (Marathi and
English), FastAPI backend, PostgreSQL, Redis. Roles: citizen, business,
officer, admin. Principles: do not replace department systems; every cross-
department send needs stored consent and an audit entry; one tracking ID per
application mapped to many department reference IDs; roles enforced on the
server. Keep the code simple, commented, and runnable with Docker Compose.
Ask me before adding any library not in this list.
```

Prompt 1. Backend scaffold and database

```
Create a FastAPI project with PostgreSQL via SQLAlchemy and Alembic. Tables: users(id, name, role, department, password_hash), applications(tracking_id, user_id, service_type, status, created_at), dept_tasks(id, tracking_id, dept, dept_ref_id, state, sla_due, warned, breached, updated_at), documents(id, tracking_id, type, validation_status), consents(id, user_id, dept, purpose, granted_at, expires_at, revoked_at), grievances(id, tracking_id, reason, escalation_level, created_at), audit_log(id, actor, action, entity, timestamp, prev_hash, hash), api_health(connector, status, latency_ms, last_error, checked_at). Add a docker-compose.yml with api, postgres and redis, a .env.example, and a /health endpoint. Include seed data for one user per role.
```

Prompt 2. Mock department systems

```
Create four small mock departments, each with a DIFFERENT data format: Industries (nested JSON), Pollution Control (flat JSON with different field names), Fire (XML), and Labour (no API, it only reads a CSV file placed in a folder). Each exposes POST /applications to receive an application and returns its own reference ID, GET /applications/{ref} for status, and a PATCH endpoint an officer can call to change state. Add an environment flag FAIL_NEXT=true that makes the next call return a 503, then works again, so I can demo retries. Each department must treat a repeated idempotency key as the same application and return the same reference ID, never a duplicate. Use synthetic data only.
```

Prompt 3. Connectors and common data model

```
Define a canonical application schema (applicant, project details, documents, requested service). For each mock department write a connector class with three methods: to_canonical(raw), from_canonical(canonical) and send(canonical). Add an API gateway module that picks the connector by department name. Add retry with exponential backoff using Redis, a dead-letter queue after 3 failures, and write each call's status and latency to api_health. Write unit tests for each mapping.
```

Prompt 4. Auth, roles and consent

```
Add JWT login with roles citizen, business, officer, admin and a dependency
that enforces roles on every route. Add POST /consent and GET /consent for
the logged-in user, storing department, purpose, timestamps and an expiry,
and support revoking. Add a function that raises an error if a send to a
department has no active consent for that user. Write an audit_log entry for
login, consent changes and every cross-department send. Make entries hash-
chained (each row stores the previous row's hash) and add a verify function
that detects any edited row.
```

Prompt 5. Checklist and validation

```
Create an editable approvals_rules.json mapping service type, sector,
location and project size to the list of required approvals and documents.
Add POST /assistant/checklist that reads it and returns the required
approvals with a plain-language reason for each. Add POST /validate that
checks required fields and document types and flags inconsistencies across
fields (for example name or address mismatch between two documents),
returning field-level messages in English and Marathi.
```

Prompt 6. Applications, workflow and SLA

```
Add POST /applications that creates a tracking_id, checks consent, then fans
out one task per required department in parallel through the gateway, saving
each dept_ref_id in dept_tasks. Send an idempotency key (tracking_id plus
department) with every call so retries never create duplicates. Derive the
application status: approved if all tasks are approved, action_needed if any
is rejected, partly_approved if some are approved, otherwise in_progress. Add
GET /applications/{id}/timeline that merges events from all departments into
one ordered list. Add a sla.json with a deadline in hours per department and
stage. Add a background worker that every minute warns at 80 percent of the
SLA, and on breach creates a grievance automatically and raises its
escalation level. Add a risk flag that routes low-risk applications to a fast
track. Add an admin-only control that shortens SLAs to 30 seconds for demos.
```

Prompt 7. Assistant, notifications and analytics

```
Add an assistant endpoint that answers in Marathi or English about which approvals and documents a user needs, using approvals_rules.json first and an LLM API only to phrase the answer and handle free-text questions; if the LLM is unavailable, return the rules-based answer. Add in-app notifications on every status change. Add GET /analytics returning average processing time per department, count of SLA breaches, slowest stages, and API health, all computed from real timestamps in the database.
```

Prompt 8. Frontend: shell, auth and citizen flow

```
Create a React app with a language switch (Marathi and English, one translation file per language, no hard-coded text), login, role-based route guards, and a citizen dashboard. Build a NewApplication wizard with a progress bar: Describe, Checklist, Upload, Consent, Submit. Show the checklist from the assistant endpoint, live validation messages on upload, and a consent screen with one card per department showing the data shared and the purpose with Allow and Deny. After submit show the tracking ID prominently. Design for a basic mobile phone first: large buttons, single column, plain words, status shown with text and icons, not colour alone.
```

Prompt 9. Frontend: tracking, officer and admin

```
Build a Tracking page with a timeline showing every department in parallel, an SLA badge per stage, a grievance form, and the assistant chat. Build an OfficerQueue sorted by SLA due time with a countdown, a detail view, and approve, reject with reason, and request more information actions. Build an AdminDashboard with charts for average processing time per department, bottlenecks, SLA breaches and API health, each titled with the finding in plain language; add a failed-calls list with a Retry button and a searchable audit log.
```

Prompt 10. Seed data, demo switches and tests

```
Add a seed script with one citizen, one business user, one officer and one admin, one clean application and one application with a deliberately inconsistent document. Add a demo panel for the admin with two buttons: make the next department call fail once, and shorten SLAs to 30 seconds. Add end-to-end tests for the full flow: login, checklist, validation error, consent, submit, parallel routing, one failed call that retries and succeeds, SLA breach with grievance, officer approval. Print the timings of each step.
```

Prompt 11. Documentation and diagrams

```
Write a README with setup using Docker Compose, a table of API endpoints, an architecture diagram and a user flow diagram in Mermaid, the connector contract, how to add a new department, and a short section stating that all departments are mocks with synthetic data and that real integration needs an MoU and authorised pilot access.
```

How to use the prompts well

- Send one prompt at a time and run it before the next; fix errors in the same chat.
- If a result is wrong, paste the exact error and ask for a fix rather than rewriting the prompt.
- Keep a note of every failure and its fix: this becomes your one honest failure for template item 06.
- Name the AI tool you used in the deck and say exactly what it did, as the guide requires.
- Do not paste real personal data or real credentials into any prompt.

10. Build guide

Build in five phases. Each phase has a "done when" test; do not start the next phase until it passes. Use section 9's prompts for the code and this section for order, structure and checks.

Project structure

```
mahasetu/
  docker-compose.yml
  backend/app/
    main.py  db.py  models.py  auth.py  consent.py  audit.py
    gateway.py  workflow.py  sla.py  assistant.py  validation.py
    rules/approvals_rules.json  rules/sla.json
    connectors/base.py  industries.py  pollution.py  fire.py  filedrop.py
  mock_depts/industries  pollution  fire  filedrop
  frontend/src/pages  components  i18n/mr.json  i18n/en.json
  tests/
  evidence/   (interviews, timings, failure log)
```

Setup commands

```
git init mahasetu && cd mahasetu
docker compose up --build          # api, postgres, redis, mock departments
cd backend && pip install -r requirements.txt && pytest
npm create vite@latest frontend -- --template react
cd frontend && npm install i18next react-i18next react-router-dom
@tanstack/react-query
```

Phase 1: Foundation (mock departments and connectors)

1. Build the four mock departments (nested JSON, flat JSON, XML, CSV file drop) with the FAIL_NEXT switch.
2. Write the connector base class and one connector per department.
3. Add the gateway with retry, dead-letter queue and api_health logging.

Done when: one canonical application sent to all four departments arrives in each one's own format, and a forced failure retries and succeeds.

Phase 2: Trust (auth, consent, audit)

1. JWT login with four roles, enforced on the server.
2. Consent records with department, purpose, start, expiry and revoke.
3. Hash-chained audit log with a verify function.

Done when: a send without consent is blocked, a revoked consent stops later sends, and editing one audit row makes verification fail.

Phase 3: Core flow (tracking, routing, partial failure)

1. One tracking ID per application, with the idempotency key tracking_id plus department on every send.
2. Parallel fan-out and a merged timeline.
3. Derived application status: all approved is approved, any rejected is action needed, otherwise in progress.

Done when: retrying the same send creates no duplicate in any department, and one rejection leaves the other approvals intact with the application shown as action needed.

Phase 4: Reliability and visibility (SLA, officer, admin)

1. SLA worker: warning at 80 percent, grievance and escalation on breach.
2. Officer queue sorted by due time.
3. Admin dashboard: average time per department, breaches, API health, failed-call retry.

Done when: an SLA shortened to 30 seconds produces a warning and then an automatic grievance, and the admin can retry a dead-lettered call.

Phase 5: Intelligence and experience (rules, validation, assistant, UI)

1. Rules-based checklist and validation with an identity-match rule and Marathi and English messages.
2. Assistant grounded in the rules file, citing the rule, saying "not sure" when nothing matches.
3. Frontend: citizen wizard, tracking page, officer and admin screens, language switch.

Done when: the assistant answers a 30 to 50 question test set and you have recorded its accuracy, and five real users complete an application and you have recorded where they got stuck.

Key code patterns

These are reference sketches in Python. Adapt names to your models, and let your coding tool fill in the rest.

Connector contract, the same for every department:

```
from abc import ABC, abstractmethod

class Connector(ABC):
    name: str

    @abstractmethod
    def to_canonical(self, raw): ...

    @abstractmethod
    def from_canonical(self, canonical): ...

    @abstractmethod
    def send(self, payload, idempotency_key: str) -> dict: ...
```

Idempotent receive in each mock department, so a retry never creates a duplicate:

```
seen = {}  # idempotency_key -> reference id

def receive(payload, key):
    if key in seen:
        return {"ref": seen[key], "duplicate": True}
    ref = new_reference_id()
    seen[key] = ref
    store(ref, payload)
    return {"ref": ref, "duplicate": False}
```

Send with consent check, audit entry and retry. The key is tracking ID plus department:

```
import time

def send_to_department(db, user_id, tracking_id, dept, canonical):
    require_consent(db, user_id, dept, purpose="application")
    conn = get_connector(dept)
    key = f"{tracking_id}:{dept}"
    for attempt in range(3):
        try:
            result = conn.send(conn.from_canonical(canonical), key)
            add_audit(db, user_id, "send_ok", f"{tracking_id}:{dept}")
            return result
        except Exception as err:
            log_health(db, dept, "error", str(err))
            time.sleep(2 ** attempt)
    move_to_dead_letter(db, tracking_id, dept)
    add_audit(db, user_id, "send_failed", f"{tracking_id}:{dept}")
```

Consent check. The consents table includes an expires_at column (see section 8 and prompt 1):

```
from datetime import datetime
from fastapi import HTTPException

def require_consent(db, user_id, dept, purpose):
    c = db.query(Consent).filter_by(
        user_id=user_id, dept=dept, purpose=purpose, revoked_at=None).first()
    if not c or (c.expires_at and c.expires_at < datetime.utcnow()):
        raise HTTPException(403, "Consent required")
```

Hash-chained audit log, where each row includes the hash of the previous row:

```
import hashlib, json
ZERO = "0" * 64

def _digest(actor, action, entity, prev):
    body = json.dumps({"actor": actor, "action": action,
                       "entity": entity, "prev": prev}, sort_keys=True)
    return hashlib.sha256(body.encode()).hexdigest()

def add_audit(db, actor, action, entity):
    last = db.query(Audit).order_by(Audit.id.desc()).first()
    prev = last.hash if last else ZERO
    db.add(Audit(actor=actor, action=action, entity=entity,
                 prev_hash=prev, hash=_digest(actor, action, entity, prev)))
    db.commit()

def verify_chain(db):
    prev = ZERO
    for r in db.query(Audit).order_by(Audit.id):
        if r.prev_hash != prev or r.hash != _digest(r.actor, r.action,
r.entity, prev):
            return False, r.id
        prev = r.hash
    return True, None
```

Write audit rows one at a time (a lock or a serial worker), otherwise two writers can read the same previous hash.

Derived application status, which handles partial failure:

```
def application_status(task_states):
    if all(s == "approved" for s in task_states): return "approved"
    if any(s == "rejected" for s in task_states): return "action_needed"
    if any(s == "approved" for s in task_states): return "partly_approved"
    return "in_progress"
```

SLA worker logic, run every minute:

```
for task in open_tasks(db):
    used = (now() - task.created_at) / (task.sla_due - task.created_at)
    if used >= 1 and not task.breached:
        task.breached = True
        create_grievance(db, task.tracking_id, reason="SLA breach")
        add_audit(db, "system", "sla_breach", str(task.id))
    elif used >= 0.8 and not task.warned:
        task.warned = True
        notify_officer(task)
```

Tests that prove the hard parts

```
def test_retry_creates_no_duplicate():
    a = dept.receive(payload, key="T1:industries")
    b = dept.receive(payload, key="T1:industries")
    assert a["ref"] == b["ref"] and b["duplicate"]

def test_send_without_consent_is_blocked(db, user):
    with pytest.raises(HTTPException):
        send_to_department(db, user.id, "T1", "fire", canonical)

def test_audit_tamper_is_detected(db):
    add_audit(db, "u1", "login", "u1"); add_audit(db, "u1", "send_ok", "T1")
    db.query(Audit).first().action = "nothing"; db.commit()
    ok, bad_id = verify_chain(db)
    assert not ok

def test_one_rejection_marks_action_needed():
    assert application_status(["approved", "rejected", "in_progress"]) ==
"action_needed"
```

Evidence to keep while building. Save these in the evidence folder; they feed the deck.

- A failure log: each real failure, what caused it, and the fix. Pick one for template item 06.
- Timings: processing time with and without the platform, from your own test runs.
- Assistant test set results: accuracy and how often it correctly said it was not sure.
- Usability notes from five real users: completion rate, time, where they got stuck, what you changed.
- Names and purpose of every tool you used, including the AI coding tool.

Before you call the build finished

- [ ] Phase "done when" tests all pass from a clean `docker compose up`.
- [ ] The four tests above run in one `pytest` command.
- [ ] The demo survives a forced department failure.
- [ ] No real personal data or credentials exist anywhere in the repo.
- [ ] README states that all departments are mocks with synthetic data.

11. Demo story and fail-safes

This is the full demo story with its fail-safes. It uses the four mock departments, the data model in section 8 and the build phases in section 10.

Demo story: Ravi starts a small factory

Use synthetic data only. Times shown in the dashboard come from your real test run; do not type them in.

1. **Log in once.** Ravi signs in and sees the citizen portal.
2. **Describe the project.** Factory name, district, size (small or medium), sector. The app calls POST /assistant/checklist.
3. **Checklist.** The assistant lists the approvals needed (Industries, Pollution Control, Fire), with the rule each came from and the documents required, in Marathi or English.
4. **Upload documents.** Ravi uploads synthetic documents: ID proof, address proof, land ownership. Each synthetic document carries pre-extracted fields, so no OCR is needed for the demo.
5. **Validation catches a problem.** The address on the ID proof does not match the address proof. Ravi corrects it and validation passes.
6. **Consent.** One card per department shows the data shared and the purpose. Ravi allows all three. The consent is stored with an expiry and logged.
7. **Submit once.** A tracking ID is created and tasks fan out in parallel to the three departments through their connectors, each in its own format.
8. **One timeline.** Ravi sees each department's status in one place.
9. **Failure and retry.** The admin switches on the failure for the Pollution Control mock. The first send fails, the retry succeeds, and the admin panel shows the failure and the retry. No duplicate application appears in Pollution Control, because of the idempotency key.
10. **SLA warning and escalation.** The admin shortens the Pollution Control SLA to 30 seconds. At 80 percent the officer gets a warning; at the deadline a grievance is created and escalated automatically.

11. **Partial outcome.** Industries approves, Fire rejects with a reason, Pollution Control approves. The application shows "action needed" for Fire only, and Ravi fixes and resubmits to Fire without redoing the others.
12. **Officer approval.** The Pollution Control officer approves from their queue and Ravi is notified.
13. **Audit.** The admin opens the audit log for the tracking ID, then runs the verify command to show the chain is intact. Optionally edit one row to show verification fail.
14. **Numbers.** The admin dashboard shows the real timings from this run.

Fail-safes (keep from the reference document)

- Seed data: one clean account and one account with a deliberately mismatched address.
- A failure switch for one mock department that fails the next call once.
- An SLA fast-forward control (30 seconds).
- A backup video of the whole flow, recorded beforehand.